Universidad Autónoma de Yucatán

Facultad de Matemáticas

Ejercicio 1: Encadenamiento en la biblioteca

Inteligencia Artificial

Fernando Alberto Loza Sánchez

6 de noviembre de 2026

Índice

1. El problema en pocas palabras **2**

2. El archivo de la base **2**

3. El código **3**
3.1. Cómo represento un átomo . . . 3
3.2. Unificar: hacer que dos átomos "encajen" . . . 3
3.3. Renombrar variables . . . 4
3.4. Hacia adelante . . . 4
3.5. Hacia atrás . . . 5

4. Las trazas **6**
4.1. Hacia adelante . . . 6
4.2. Hacia atrás . . . 7
4.3. Resumen . . . 8

5. Reporte **8**
5.1. ¿Por qué `Becario(Ana)` sale hacia adelante y no hacia atrás? . . . 8
5.2. ¿Por qué hacia atrás prueba `R_miembro_est` con Marta, si es profesora? . . . 8
5.3. ¿Qué pasa si se añade `TieneCredencial(Luis)`? . . . 8

1. El problema en pocas palabras

Tengo una base de conocimiento con **hechos** (cosas que sé que son ciertas) y **reglas** de la forma "si pasa esto y esto, entonces pasa aquello". Todas son *cláusulas definidas*: varias premisas y *una sola* conclusión. La pregunta es: ¿quién puede entrar de noche a la biblioteca?

Hechos: `Estudiante(Ana)`, `Estudiante(Luis)`, `Profesor(Marta)`, `TieneCredencial(Ana)`, `TieneCredencial(Marta)`, `HorarioNocturno(Hoy)`.

Reglas (en este orden, que importa):

Id	Si	Entonces
`R_miembro_est`	`Estudiante(x)` y `TieneCredencial(x)`	`Miembro(x)`
`R_miembro_prof`	`Profesor(x)`	`Miembro(x)`
`R_acceso`	`Miembro(x)` y `HorarioNocturno(Hoy)`	`AccesoNoche(x)`
`R_beca`	`Estudiante(x)` y `TieneCredencial(x)`	`Becario(x)`

En palabras: eres *miembro* si eres estudiante con credencial, o si eres profesor. Si eres miembro y hoy hay horario nocturno, tienes acceso de noche. Y aparte (sin que tenga que ver con el acceso), un estudiante con credencial es becario.

Una convención que hay que tener en la cabeza: **minúscula = variable** (`x` puede ser cualquiera), **mayúscula = constante** (`Ana`, `Hoy` son cosas concretas).

Hay dos formas de contestar:

- **Hacia adelante** (dirigido por los *datos*): parto de los hechos y disparo toda regla que pueda, una y otra vez, hasta que ya no salga nada nuevo. Al final me fijo si la consulta quedó entre los hechos.
- **Hacia atrás** (dirigido por la *meta*): parto de la consulta y me pregunto "¿qué regla podría concluir esto?". Eso me da nuevas metas (sus premisas), y así hasta llegar a hechos.

2. El archivo de la base

Lo escribí en YAML con el mismo formato que el proyecto de la clase (`1.biblioteca.yaml`). Le agregué la lista `queries` con las tres consultas:

```
facts:
  - Estudiante(Ana)
  - Estudiante(Luis)
  - Profesor(Marta)
  - TieneCredencial(Ana)
  - TieneCredencial(Marta)
  - HorarioNocturno(Hoy)

rules:
  - id: R_miembro_est
    if: [Estudiante(x), TieneCredencial(x)]
    then: Miembro(x)
  - id: R_miembro_prof
    if: [Profesor(x)]
    then: Miembro(x)
  - id: R_acceso
    if: [Miembro(x), HorarioNocturno(Hoy)]
    then: AccesoNoche(x)
  - id: R_beca
    if: [Estudiante(x), TieneCredencial(x)]
    then: Becario(x)

queries:
  - AccesoNoche(Ana)
  - AccesoNoche(Luis)
  - AccesoNoche(Marta)
```

3. El código

Todo está en `1.encadenamiento.py`, en Python puro (solo uso `yaml` para leer la base). Se corre así:

```
python3 1.encadenamiento.py 1.biblioteca.yaml
```

3.1. Cómo represento un átomo

Un átomo como `Miembro(Ana)` lo guardo como una tupla `("Miembro", (.Ana",))`: el predicado y la lista de argumentos. Así puedo compararlos con `==` y buscarlos con `in`.

```
def leer_atomo(texto):
    """'Miembro(x)' -> ('Miembro', ('x',))"""
    texto = texto.strip()
    predicado, resto = texto.split("(", 1)
    argumentos = resto.rstrip(")").split(",")
    return (predicado.strip(), tuple(a.strip() for a in argumentos))


def a_texto(atomo):
    """('Miembro', ('Ana',)) -> 'Miembro(Ana)'"""
    predicado, args = atomo
    return f"{predicado}({', '.join(args)})"


def es_variable(termino):
    """Las variables empiezan con minuscula: x, y, x_3."""
    return termino[0].islower()
```

3.2. Unificar: hacer que dos átomos "encajen"

Esta es la pieza clave de los dos algoritmos. **Unificar** `Estudiante(x)` con `Estudiante(Ana)` quiere decir: ¿qué valor le tengo que dar a `x` para que sean iguales? Respuesta: `x = Ana`. Eso se guarda en un diccionario llamado `theta` (la *sustitución*): {"x": .Ana"}.

```
def sustituir(atomo, theta):
    """Cambia cada variable por su valor en theta (si lo tiene)."""
    predicado, args = atomo
    nuevos = []
    for a in args:
        while es_variable(a) and a in theta: # sigue la cadena x -> y -> Ana
            a = theta[a]
        nuevos.append(a)
    return (predicado, tuple(nuevos))


def unificar(a1, a2, theta):
    """Intenta hacer iguales a1 y a2. Regresa la sustitucion extendida o None."""
    if theta is None:
        return None
    a1 = sustituir(a1, theta)
    a2 = sustituir(a2, theta)
    if a1[0] != a2[0] or len(a1[1]) != len(a2[1]):
        return None # distinto predicado o aridad
    theta = dict(theta) # copia: no ensuciar la de afuera
    for t1, t2 in zip(a1[1], a2[1]):
        t1 = sustituir(("", (t1,)), theta)[1][0]
        t2 = sustituir(("", (t2,)), theta)[1][0]
        if t1 == t2:
            continue
        if es_variable(t1):
            theta[t1] = t2
        elif es_variable(t2):
```

```
            theta[t2] = t1
        else:
            return None # dos constantes distintas: Ana != Luis
    return theta
```

Casos:

- `Estudiante(x)` con `Estudiante(Ana)` → {`x: Ana`}.
- `Estudiante(Ana)` con `Profesor(Marta)` → `None` (distinto predicado).
- `Estudiante(Ana)` con `Estudiante(Luis)` → `None` (dos constantes distintas).
- Si `theta` ya dice `x = Ana`, entonces `TieneCredencial(x)` se lee como `TieneCredencial(Ana)`, y *solo* unifica con ese hecho. Así las premisas de una regla hablan de la misma persona.

3.3. Renombrar variables

Cada vez que uso una regla le cambio el nombre a sus variables: `x` pasa a `x_1`, `x_2`, etc. Es para que dos usos de reglas distintas (por ejemplo `R_acceso` y, adentro, `R_miembro_est`) no compartan la misma `x` por accidente. En la traza imprimo solo {`x/Ana`}, sin el número.

```
def renombrar(regla, n):
    """Copia de la regla con variables nuevas: x -> x_n (para que dos usos no choquen)."""
    nombre, premisas, cabeza = regla

    def ren(atomo):
        pred, args = atomo
        return (pred, tuple(f"{a}_{n}" if es_variable(a) else a for a in args))

    return nombre, [ren(p) for p in premisas], ren(cabeza)
```

3.4. Hacia adelante

```
def satisfacer(premisas, hechos, theta):
    """Todas las sustituciones que hacen verdaderas TODAS las premisas, en orden."""
    if not premisas:
        return [theta]
    primera, resto = premisas[0], premisas[1:]
    resultados = []
    for hecho in hechos: # los hechos en el orden en que estan
        theta2 = unificar(primera, hecho, theta)
        if theta2 is not None:
            resultados.extend(satisfacer(resto, hechos, theta2))
    return resultados
```

`satisfacer` encuentra *todas* las formas de hacer verdaderas las premisas de una regla: toma la primera premisa, la intenta unificar con cada hecho (en el orden en que están), y con cada `theta` que funcione sigue con el resto de las premisas.

```
def hacia_adelante(hechos_iniciales, reglas, consulta, max_rondas=50):
    hechos = list(hechos_iniciales)
    nuevos_total = []
    traza = [f"Hechos iniciales: {', '.join(a_texto(h) for h in hechos)}",
             f"Consulta: {a_texto(consulta)}"]
    contador = 0

    for ronda in range(1, max_rondas + 1):
        traza.append(f"--- ronda {ronda} ---")
        nuevos = [] # lo que sale en ESTA ronda
        for regla in reglas: # las reglas en el orden de la tabla
            contador += 1
            nombre, premisas, cabeza = renombrar(regla, contador)
```

```
            # OJO: uso 'hechos' (lo que habia al empezar la ronda), no 'nuevos'
            for theta in satisfacer(premisas, hechos, {}):
                conclusion = sustituir(cabeza, theta)
                if conclusion in hechos or conclusion in nuevos:
                    continue # ya lo sabia: no lo repito
                nuevos.append(conclusion)
                traza.append(f"Dispara {nombre} {sustitucion_bonita(theta)} => {a_texto(conclusion)}")
        if not nuevos:
            traza.append("Nada nuevo: se detiene.")
            break
        hechos.extend(nuevos) # hasta AHORA se pueden usar como premisas
        nuevos_total.extend(nuevos)

    # La consulta se busca al final, entre todo lo que quedo
    probada = False
    for hecho in hechos:
        if unificar(consulta, hecho, {}) is not None:
            probada = True
            break
    traza.append(f"Atomos nuevos: {', '.join(a_texto(a) for a in nuevos_total)}")
    traza.append("RESULTADO: probada" if probada else "RESULTADO: NO probada")
    return probada, nuevos_total, traza
```

Las tres cosas que pide el enunciado están en el código:

1. **Rondas.** Dentro de una ronda, las reglas solo ven `hechos` (lo que había al empezar la ronda). Lo que sale se guarda en `nuevos` y se pega a `hechos` hasta que termina la ronda (`hechos.extend(nuevos)`).
2. **Sin repetidos.** Si la conclusión ya está en `hechos` o en `nuevos`, no se vuelve a añadir.
3. **Hasta que no haya nada nuevo.** El ciclo solo para cuando una ronda no produce nada, aunque la consulta ya se hubiera probado antes. La consulta se revisa *al final*.

3.5. Hacia atrás

```
def hacia_atras(hechos, reglas, consulta, max_prof=25):
    traza = [f"Consulta: {a_texto(consulta)}"]
    contador = [0]

    def sangria(p):
        return " " * p

    def probar_meta(meta, theta, prof, pila):
        """bc_or: probar UNA meta. Primero hechos, luego reglas. Regresa theta o None."""
        meta = sustituir(meta, theta)
        if a_texto(meta) in pila:
            traza.append(f"{sangria(prof)}ciclo: {a_texto(meta)} ya esta en la pila, se corta")
            return None
        if prof > max_prof:
            traza.append(f"{sangria(prof)}demasiado profundo en {a_texto(meta)}")
            return None
        traza.append(f"{sangria(prof)}meta {a_texto(meta)}")

        for hecho in hechos: # 1) es un hecho?
            theta2 = unificar(meta, hecho, theta)
            if theta2 is not None:
                traza.append(f"{sangria(prof)}hecho {a_texto(hecho)}")
                return theta2

        for regla in reglas: # 2) alguna regla concluye la meta?
            contador[0] += 1
            nombre, premisas, cabeza = renombrar(regla, contador[0])
            theta2 = unificar(meta, cabeza, theta)
            if theta2 is None:
```

```
            continue # la cabeza no se parece: ni la abro
        traza.append(f"{sangria(prof)}regla {nombre} {sustitucion_bonita(theta2)}")
        respuesta = probar_todas(premisas, theta2, prof + 1, pila + [a_texto(meta)])
        if respuesta is not None:
            return respuesta # la primera que funciona gana
    traza.append(f"{sangria(prof)}falla {a_texto(meta)}")
    return None

def probar_todas(premisas, theta, prof, pila):
    """bc_and: probar las premisas una tras otra; si una falla, falla todo."""
    if not premisas:
        return theta
    theta2 = probar_meta(premisas[0], theta, prof, pila)
    if theta2 is None:
        return None
    return probar_todas(premisas[1:], theta2, prof, pila)

resultado = probar_meta(consulta, {}, 0, [])
probada = resultado is not None
traza.append("RESULTADO: probada" if probada else "RESULTADO: NO probada")
return probada, traza
```

Son dos funciones que se llaman una a la otra:

- `probar_meta` (el `bc_or` del proyecto): "¿cómo pruebo *esta* meta?" Primero busca un hecho que la iguale. Si no hay, recorre las reglas en el orden de la tabla y solo *abre* las que tienen una cabeza que unifica con la meta (las demás ni aparecen en la traza). Para cada regla abierta intenta probar sus premisas; **la primera que funciona gana** y ya no prueba las demás. Si nada funciona: `falla`.
- `probar_todas` (el `bc_and`): prueba las premisas en orden, pasando la `theta` de una a la siguiente. Si una falla, falla todo.
- La `pila` guarda las metas que están "abiertas" en el camino actual. Si una meta ya está ahí, se corta (sería un ciclo). En esta base no pasa, pero la protección está.
- Nunca se agrega nada a `hechos`: hacia atrás *no* deriva átomos, solo comprueba.

4. Las trazas

La salida completa también está en `2.trazas.txt`.

4.1. Hacia adelante

Hacia adelante no depende de la consulta para derivar: las tres corridas hacen *exactamente* lo mismo y solo cambia la revisión del final. Esta es la de `AccesoNoche(Ana)`:

```
Hechos iniciales: Estudiante(Ana), Estudiante(Luis), Profesor(Marta), TieneCredencial(Ana),
    TieneCredencial(Marta), HorarioNocturno(Hoy)
Consulta: AccesoNoche(Ana)
--- ronda 1 ---
Dispara R_miembro_est {x/Ana} => Miembro(Ana)
Dispara R_miembro_prof {x/Marta} => Miembro(Marta)
Dispara R_beca {x/Ana} => Becario(Ana)
--- ronda 2 ---
Dispara R_acceso {x/Ana} => AccesoNoche(Ana)
Dispara R_acceso {x/Marta} => AccesoNoche(Marta)
--- ronda 3 ---
Nada nuevo: se detiene.
Atomos nuevos: Miembro(Ana), Miembro(Marta), Becario(Ana), AccesoNoche(Ana), AccesoNoche(Marta)
RESULTADO: probada
```

Las de `AccesoNoche(Luis)` y `AccesoNoche(Marta)` tienen las mismas rondas y los mismos 5 átomos nuevos; solo cambia la última línea:

consulta	resultado hacia adelante
`AccesoNoche(Ana)`	probada
`AccesoNoche(Luis)`	**NO** probada
`AccesoNoche(Marta)`	probada

Cómo leerla:

- **Ronda 1.** Con los hechos originales disparan `R_miembro_est` (Ana), `R_miembro_prof` (Marta) y `R_beca` (Ana). `R_acceso` *no* dispara todavía: necesita `Miembro(...)`, y eso aún no está en los hechos de esta ronda.
- **Ronda 2.** Ya con `Miembro(Ana)` y `Miembro(Marta)`, dispara `R_acceso` para las dos.
- **Ronda 3.** Nada nuevo: se detiene.
- Luis nunca aparece en un átomo nuevo: es estudiante pero no tiene credencial, así que no es miembro ni becario.

4.2. Hacia atrás

`AccesoNoche(Ana)`: probada.

```
Consulta: AccesoNoche(Ana)
meta AccesoNoche(Ana)
regla R_acceso {x/Ana}
  meta Miembro(Ana)
  regla R_miembro_est {x/Ana}
    meta Estudiante(Ana)
    hecho Estudiante(Ana)
    meta TieneCredencial(Ana)
    hecho TieneCredencial(Ana)
  meta HorarioNocturno(Hoy)
  hecho HorarioNocturno(Hoy)
RESULTADO: probada
```

Se prueba con `R_acceso` y `R_miembro_est`, apoyada en los hechos `Estudiante(Ana)`, `TieneCredencial(Ana)` y `HorarioNocturno(Hoy)`. No aparecen Luis, Marta, `R_beca` ni `Becario`.

`AccesoNoche(Luis)`: NO probada.

```
Consulta: AccesoNoche(Luis)
meta AccesoNoche(Luis)
regla R_acceso {x/Luis}
  meta Miembro(Luis)
  regla R_miembro_est {x/Luis}
    meta Estudiante(Luis)
    hecho Estudiante(Luis)
    meta TieneCredencial(Luis)
    falla TieneCredencial(Luis)
  regla R_miembro_prof {x/Luis}
    meta Profesor(Luis)
    falla Profesor(Luis)
  falla Miembro(Luis)
falla AccesoNoche(Luis)
RESULTADO: NO probada
```

`R_miembro_est` falla porque `TieneCredencial(Luis)` no es hecho, y `R_miembro_prof` falla porque `Profesor(Luis)` tampoco. Sin `Miembro(Luis)`, `R_acceso` no se puede usar, y no hay otra regla que concluya `AccesoNoche`.

`AccesoNoche(Marta)`: probada.

```
Consulta: AccesoNoche(Marta)
meta AccesoNoche(Marta)
```

```
regla R_acceso {x/Marta}
  meta Miembro(Marta)
  regla R_miembro_est {x/Marta}
    meta Estudiante(Marta)
    falla Estudiante(Marta)
  regla R_miembro_prof {x/Marta}
    meta Profesor(Marta)
    hecho Profesor(Marta)
  meta HorarioNocturno(Hoy)
  hecho HorarioNocturno(Hoy)
RESULTADO: probada
```

Primero intenta `R_miembro_est`, falla porque no existe `Estudiante(Marta)`, y entonces prueba `R_miembro_prof` con `Profesor(Marta)`.

4.3. Resumen

consulta	hacia adelante	hacia atrás
`AccesoNoche(Ana)`	sí	sí
`AccesoNoche(Luis)`	no	no
`AccesoNoche(Marta)`	sí	sí

Los dos algoritmos dan la misma respuesta (tienen que darla: los dos son correctos y completos para cláusulas definidas). Lo que cambia es *cuánto trabajo hacen* para llegar a ella.

5. Reporte

5.1. ¿Por qué `Becario(Ana)` sale hacia adelante y no hacia atrás?

Porque hacia adelante **no mira la consulta mientras deriva**: dispara *toda* regla cuyas premisas ya son hechos. En la ronda 1, `Estudiante(Ana)` y `TieneCredencial(Ana)` ya son hechos, así que `R_beca` dispara y sale `Becario(Ana)`, aunque a nadie le importe para el acceso. Es un átomo "de más".

Hacia atrás **solo abre reglas cuya cabeza unifica con la meta actual**. Las metas que aparecen son `AccesoNoche(Ana)`, `Miembro(Ana)`, `Estudiante(Ana)`, `TieneCredencial(Ana)` y `HorarioNocturno(Hoy)`. La cabeza de `R_beca` es `Becario(x)`, que no unifica con ninguna de ellas (distinto predicado), así que esa regla nunca se abre.

5.2. ¿Por qué hacia atrás prueba `R_miembro_est` con Marta, si es profesora?

Porque el algoritmo **no sabe** que Marta es profesora hasta que lo comprueba. Al buscar `Miembro(Marta)`, mira las reglas en el orden de la tabla y se queda con las que tienen cabeza `Miembro(x)`. Hay dos, y `R_miembro_est` está primero. Su cabeza unifica con `x = Marta`, así que la intenta: su primera premisa, `Estudiante(Marta)`, no es hecho y ninguna regla la concluye, así que falla. Solo entonces pasa a la siguiente, `R_miembro_prof`, que sí funciona con el hecho `Profesor(Marta)`.

Es decir: la decisión de qué regla abrir se toma por la *cabeza* (lo que concluye), no por las premisas. Si en la tabla `R_miembro_prof` estuviera primero, Marta se probaría sin pasar por el fallo (y Ana tendría que fallar primero en la de profesor).

5.3. ¿Qué pasa si se añade `TieneCredencial(Luis)`?

Lo corrí con mi programa (el hecho se agrega en memoria, al final de la lista; el YAML no se toca).

Hacia adelante:

```
Hechos iniciales: Estudiante(Ana), Estudiante(Luis), Profesor(Marta), TieneCredencial(Ana),
    TieneCredencial(Marta), HorarioNocturno(Hoy), TieneCredencial(Luis)
Consulta: AccesoNoche(Luis)
--- ronda 1 ---
Dispara R_miembro_est {x/Ana} => Miembro(Ana)
Dispara R_miembro_est {x/Luis} => Miembro(Luis)
Dispara R_miembro_prof {x/Marta} => Miembro(Marta)
Dispara R_beca {x/Ana} => Becario(Ana)
Dispara R_beca {x/Luis} => Becario(Luis)
--- ronda 2 ---
Dispara R_acceso {x/Ana} => AccesoNoche(Ana)
Dispara R_acceso {x/Luis} => AccesoNoche(Luis)
Dispara R_acceso {x/Marta} => AccesoNoche(Marta)
--- ronda 3 ---
Nada nuevo: se detiene.
Atomos nuevos: Miembro(Ana), Miembro(Luis), Miembro(Marta), Becario(Ana), Becario(Luis), AccesoNoche(
    Ana), AccesoNoche(Luis), AccesoNoche(Marta)
RESULTADO: probada
```

Aparecen **tres átomos nuevos de Luis**: `Miembro(Luis)` y `Becario(Luis)` en la ronda 1, y `AccesoNoche(Luis)` en la ronda 2. En total son 8 átomos nuevos (los 5 de antes más esos 3), y `AccesoNoche(Luis)` queda **probada**.

Hacia atrás:

```
Consulta: AccesoNoche(Luis)
meta AccesoNoche(Luis)
regla R_acceso {x/Luis}
  meta Miembro(Luis)
  regla R_miembro_est {x/Luis}
    meta Estudiante(Luis)
    hecho Estudiante(Luis)
    meta TieneCredencial(Luis)
    hecho TieneCredencial(Luis)
  meta HorarioNocturno(Hoy)
  hecho HorarioNocturno(Hoy)
RESULTADO: probada
```

Ahora `R_miembro_est` sí funciona con Luis (los dos hechos existen) y la consulta queda **probada**, sin pasar por `R_miembro_prof` y sin derivar `Becario(Luis)`: otra vez hacia atrás solo hace lo necesario para la consulta.